

# **Cache Awareness, Cache Policies and Memoization**

# Table of Contents

- What is a cache and cache replacement policy?
- Application and Software Caches
- Hardware Level Caches
- MESI Protocol
- Locality and Temporality
- Memoization

# What Are Caches?

Similar to RAM/Memory, Caches play a role in everyday computing. They are used in a way that may not be apparent and yet they are also one of the biggest speedups your system and program gets when executing code outside of an algorithmic speed up.

Caches can also play a role in algorithms, while not obviously called a cache as they serve the purpose of reducing runtime, holding answers to existing calculations.

# Cache Policies and Memory Hierarchy

A cache's role within an application or system is to be an intermediary point between where data is stored/persisted and where it is used/operated on.

This typically results in a cache that is smaller than the storage medium itself. In the event your cache is larger than the intended storage volume, then if the data is not needing to be synchronised with any other participants then your cache can be considered your new storage medium.

We operate under the assumption that cache is smaller than the original storage volume.

# Caches - When to use one

When to incorporate a cache can be kind of difficult because there are a variety of cases where it makes sense.

- Latency is considered unacceptable
- Data is used frequently but updates are infrequent
- Reducing interaction
- Caching operations or data to be done a queue
- Maintaining results where computation of the result is expensive (See: Floyd-Warshall algorithm)

# CPU Caches

Within a memory system for a computer we have a memory hierarchy. Typically, this will include.

- Hard-Disk (Used for swap or persistent storage)
- RAM
- L3 Cache
- L2 Cache
- L1 Cache

Even here, the Hard-Disk has its own cache to operate on local data while data takes time to come in.

# CPU Caches - What is the significance?

Each cache level indicates a closeness to the CPU, this reduces the latency and while I would like to provide some easy unit to utilise, this is where we are in cycle-territory. A rough estimate here, based on 7-zip LZMA benchmark of Ice Lake (1065G7).

L1 Cache Latency = 5 cycles

L2 Cache Latency = 13 cycles

L3 Cache Latency = ~42 cycles

RAM Latency = Well this is a little more involved! However, a rough estimate is double of L3 (~84ns)

RAM, even with similar speeds, may have different timings where:

- Column Load response time may have  $K$  cycles until it executes
- Time between activation line and column - Storing the data to be processed, along with getting access to the line
- Activation to precharge delay - How long it will need to wait until next access
- Command Rate - The rate it takes between the memory chip having been activated and when the first command may be sent to memory

The main measure though is the column load response time as it is typically the timing mechanism that is most observable.



# Hang on! I've noticed ram timings are larger now!

Now! It may seem like a scam because some older ram modules may have lower timings than newer ones.

For example, DDR4 ram may have CL26 where DDR5 may have CL40 and yet, how is it "faster"?

This is because it is related to the frequency of the ram itself! Hypothetical, let's consider the following.

- DDR4 with CL26 and 3200mhz clock = 16.25ns
- DDR5 with CL40 and 5600mhz clock = 14.28ns

Ah! There is a difference! It is minor! We are comparing what is top of the line for DDR4 with something standard of DDR5. My current laptop has a CL33 and frequency of 8533 which results in latency of ~7.73ns.

# How did you derive that?

The formula for latency is roughly the following.

In addition, you also have clock-rate which will also impact processing of memory access as well.

$$\textit{Latency} = (\textit{CAS}/\textit{Mhz}) * 2000$$

- Latency is measured in nanoseconds
- *Why 2000?* 2000 is noting that the transfer is conducted on both rising and falling edges of the signal, the transfer rate is double the actual clock speed of ram itself. So, 8533mhz transfer rate is 4266.5mhz.
- Latency is not everything, bandwidth and the number of commands it can handle/issue/queue are also factors.

# CPU Cache Protocol

As outlined before, we have multiple caches, however we also have a protocol for the **Cache-Line**.

- A **Cache-Line** is 64 bytes

# CPU Cache Protocol

The protocol is typically described as **MESI**. This acronym outlines the following states of a cache-line.

- Modified - This state is where the cache-line itself is out of sync of main-memory, this occurs when writes happen.
- Exclusive - This state is where the cache-line itself is in sync with main-memory, this occurs when a single-core has read from memory
- Shared - This state is where the cache-line itself is in sync with main-memory, while this is similar to Exclusive, it is also where multiple cores/threads are reading from this cache-line
- Invalid - This state is where the cache-line is invalid, however this state can transition to Modified, Exclusive or Shared.

# CPU Cache Protocol - Why?

It can be strange that we want to be mindful of the cpu-cache when we already a few abstraction levels away from assembly.

This is because being aware of the cpu-cache and how caches work in general has a clear relationship with **realtime** execution of code.

In saying that, it can seem like you don't have much power because C# is an interpreted language and the bytecode it emits may leave it up to the VM to optimise.

This partially true that the VM can make these decisions but we can still structure performance oriented code with C#.

# CPU Cache Protocol - Cache-Lines

First thing we want to introduce here are **structs** in C#.

This is because a **struct**, similar a hangover from C/C++ roots of this language, has something similar to **int**, **bool**, **char**.

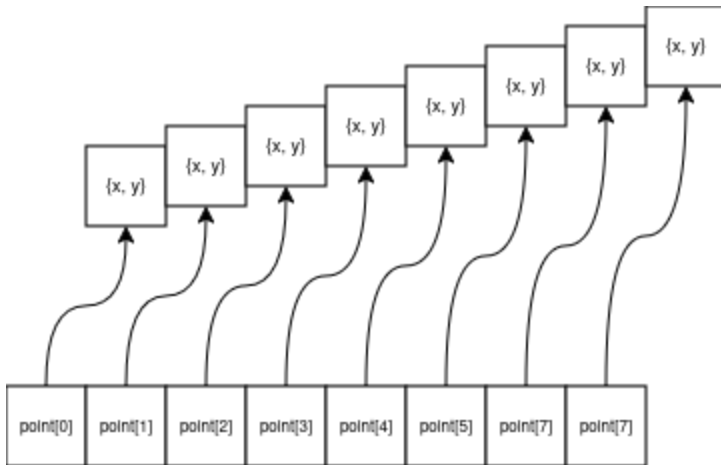
**It is a value type!**

*Why is this important?* - Because with classes, these are pointers to where the object lives. With structs, we don't hold these types as pointers, we would copy them. This is also where the **ref** and **out** keywords also find a strong sense of usage.

# CPU Cache - Memory-Access Patterns and Layout

To demonstrate effectiveness of storing data as structs over classes, we will examine how it is represented in memory.

- Here, we demonstrate how memory is represented for the array of a class object.



# CPU Cache - Memory-Access Patterns and Layout

- We are able to transform this/flatten it for structs, we can observe how it is represented here.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| {x, y} | {x, y} | {x, y} | {x, y} | {x, y} | {x, y} | {x, y} | {x, y} |
|--------|--------|--------|--------|--------|--------|--------|--------|

Given the indirection with class objects, we typically invalidate the cache while traversing the array where with struct objects, we are able to access and refer to the elements in the cache.



# Cache - Locality and Temporality

We always need the reminder that caches are smaller than ram, this means we have to understand, alongside access patterns, the amount of data we are processing and how often we would revisit it. This require considering the locality aspect of memory and how long we are using the data for.

- Locality - Is the memory we are accessing close together? Consider why indirection previous is a problem from this aspect alone. (Hint: Remember cache-lines and pre-fetching)
- Temporality - Is the memory you are reading and going to use for a final calculation going to be evicted as you find another item? It is often that with small sizes we can observe speed ups due to locality but it can fall apart when we forget how big the memory footprint is.

# Cache - Locality

- *"Have you ever wondered why summations can be incredibly quick?"*

As an experiment, try these access patterns on an int array of 1000000.

- You can only access each  $i \bmod 16 = k$  element in the array, each time you finish going through the array, you can increment  $k$  by 1, repeat this until  $k$  is 16.
- You can only access each  $i \bmod 16 = 0$  element in the array initially. After you have done this, starting with the last segment, navigate the array in reverse order.
- Access the array from 0 to  $N$  (typical way).

Time the operations and observe what time statements you get.

# Cache - Temporality

Temporality is typically ignored until we usually see a dataset bigger than our test cases.

As a thought experiment, consider the following workloads and what eviction would look like in these instances.

- Streaming Video or Audio
- Loading textures for a 3D model in some CAD software
- Operating on  $64 * 64$  byte blocks within an  $N * M$  array.

# Cache Replacement Policy

Obviously we can't store everything in cache, however, just taking a particular cache-algorithm isn't going to cut it as well.

Depending on the workload, you may want to factor in the kind cache replacement algorithm you want but also understand how eviction from a cache occurs or the choice to evict from a cache.

# Cache Replacement Policy

A common and fairly robust cache replacement algorithm is called *Least-Recently-Used*. This algorithm has been effectively the standard for a number of systems (Page-Cache, Virtual Machine Caches, Compiler Caches, etc). While it has not always yielded a optimal solution to all workload, it usually is pretty close.

The idea is fairly straight forward.

- Given a program that will access an item
  - If the item already exists in the cache, we will supply the item from the cache
  - If the item does not exist in the cache,
    - If the cache is not full, it will just be placed in the cache
    - If the cache is full, then we need to evict an item that was least recently used

# Cache Replacement Policy - FIFO Cache Exercise

Given the following scenario, you have to create a program which will read files from a hard-drive and quickly return the data when requested. This data will have to be in memory and not on the hard-disk after the first read.

We don't want read the file again if we have already read it before! If we have read it before we want to be able to pull it from a cache.

However, if the cache is small, we should dump the contents we have read from disk and only keep track of things we have read but are now considered "unloaded".

# Memoization

So, we've been talking about caches for all of this so far. How does this relate to Algorithms and Data Structures in a more formal way?

We have an optimisation technique called memoization which does employ the usage of a cache, lookup table or some other mechanism to store answers.

We keep the result as we may want to reuse it later.

# Memoization

A typical example relates to a chain of calls that would need to be performed. We can identify a set of dependencies within that chain where the last call or how we derive the answer is from the previous computation.

This is a common case where the exchange is an increase of space for a reduction of time. In practicality, this can be transformative to an algorithm if we are aware of the necessary space needed ahead of time and how often we would re-use an answer.



# Memoization/Cache - Fibonacci

While this example does not do much to transform the time-complexity, we can get a benefit from subsequent runs and real-time speed up which is proportional to the space needed to hold previous calculations.

Base Cases:

$$F_0 = 0$$

$$F_1 = 1$$

Recursive Case:

$$F_n = F_{n-1} + F_{n-2}$$

Now, let's identify the steps  $F_{n-1}$  and its recurrence relation, let's try  $F(6)$

# Memoization/Cache - Fibonacci

Let's go:

$$1. F(6) = F(5) + F(4)$$

$$2. F(6) = (F(4) + F(3)) + (F(3) + F(2))$$

$$3. F(6) = ((F(3) + F(2)) + (F(2) + F(1))) + (F(2) + F(1)) + (F(1) + F(0))$$

How about we line this up!

Interesting! Let's even show the redundant operations as well!

$$\begin{aligned} F(6) &= F(5) + F(4) \\ F(6) &= (F(4) + F(3)) + (F(3) + F(2)) \\ F(6) &= ((F(3) + F(2)) + (F(2) + F(1))) + ((F(2) + F(1)) + (F(1) + F(0))) \end{aligned}$$

Interesting! Let's even show the redundant operations as well!

$$\begin{aligned}
 F(6) &= && F(5) && + && F(4) \\
 F(6) &= && (F(4) && + && F(3)) && + && (F(3) && + && F(2)) \\
 F(6) &= && ((F(3) && + && F(2)) && + && (F(2) && + && F(1)) && + && (F(2) && + && F(1)) && + && (F(1) && + && F(0))) \\
 F(6) &= && ((F(2) && + && F(1)) && + && (1) && + && (1 + 1) && + && (1 + 1) && + && (1)) \\
 F(6) &= && (1 + 1) && + && (1) && + && (1 + 1) && + && (1 + 1) && + && (1)
 \end{aligned}$$